

AppGuard Security Guide

Automated Application Vulnerability & Hardening Audit Framework

SPECIFICATION v2.4
CLASSIFICATION: PUBLIC

1. Color Coding System: What Do the Colors Mean?

AppGuard categorizes security health, hackability risks, and findings into four standardized color domains based on OWASP MASVS and Microsoft SDL baselines:

● GREEN

PASSED

Score: 0.0 – 3.9 / 10

Verdict: Production Ready

The binary passes core memory safety, exploit mitigations, and credential hygiene. Exploit mitigation flags (ASLR/DEP) are fully engaged, no hardcoded API tokens or debugging hooks were discovered, and section entropy is normal.

Deployment Policy: Approved for staging and public production release.

● YELLOW

WARNING

Score: 4.0 – 6.9 / 10

Verdict: Needs Hardening

Moderate attack surfaces or hygiene gaps detected. The application is functional but lacks defense-in-depth measures (e.g., missing SafeSEH, ADB backup enabled, permissive exported intent filters, or exposed test endpoints).

Deployment Policy: Requires remediation of warnings before GA rollout.

● RED

CRITICAL

Score: 7.0 – 10.0 / 10

Verdict: Deployment Blocked

Immediate, high-severity vulnerabilities identified. Examples include active debuggable=true, cleartext HTTP traffic allowed, missing ASLR/DEP (vulnerable to ROP/buffer exploits), hardcoded cloud credentials, or packed malware entropy.

Deployment Policy: Immediate release freeze. Must patch before distribution.

● BLUE / CYAN

INFORMATIONAL

Represents architectural improvements, static metadata (compiler timestamp, PE machine architecture, APK target SDK, permissions inventory), and recommended best-practice optimizations that do not represent an active vulnerability.

2. Quantitative Hackability Scoring Model

The Hackability Score ranges from **1.0 (Most Secure)** to **10.0 (Trivially Exploitable)**. Scores are calculated dynamically based on finding severity:

Finding Level	Score Weight	Typical Vulnerabilities	Production Policy Gate
CRITICAL	+2.5 to +3.5	Debuggable APK enabled, Leaked AWS Keys, Missing ASLR+DEP	BLOCKER: Automatic CI/CD rejection
HIGH	+1.5 to +2.0	usesCleartextTraffic, High Section Entropy (>7.2), Unsigned binary	BLOCKER: Requires security sign-off
MEDIUM	+0.8 to +1.2	allowBackup enabled, Missing SafeSEH, Permissive exported components	WARNING: Must be resolved before v1.0
LOW	+0.2 to +0.5	Verbose logging APIs, missing Control Flow Guard, outdated target SDK	ADVISORY: Address in regular sprint backlog

Production Gate Rule: Any application with **1 or more Critical findings** or a composite score ≥ 7.0 is marked as BLOCKED. Applications with a score between **4.0 and 6.9** are marked NEEDS HARDENING. Applications with score ≤ 3.9 are marked READY.

AppGuard Technical Audit Catalog

Deep Inspection Details for Windows (.exe) & Android (.apk)

SECTION 3: CHECKS

3. Windows Executable (.exe / PE32+) Security Checks

Security Check	Default Safe	Risk if Absent	What It Does & How to Fix
ASLR (Address Space Layout Randomization)	ENABLED	CRITICAL	Randomizes virtual memory offsets at load time. If absent, attackers use deterministic memory addresses for Return-Oriented Programming (ROP) exploits. <i>Fix: Add /DYNAMICBASE linker flag.</i>
DEP / NX (Data Execution Prevention)	ENABLED	CRITICAL	Prevents execution of code located in data/stack/heap memory regions, mitigating buffer overflow shellcode attacks. <i>Fix: Add /NXCOMPAT linker flag.</i>
SafeSEH (Structured Exception Handler)	ENABLED	MEDIUM	Maintains a table of verified exception handler entrypoints to prevent SEH overwrite vulnerabilities (32-bit x86). <i>Fix: Add /SAFESEH linker flag.</i>
Control Flow Guard (CFG)	ENABLED	LOW	Verifies that indirect function calls target legitimate compiled functions rather than arbitrary injected code. <i>Fix: Add /guard:cf compiler/linker flag.</i>
Digital Signature (Authenticode)	SIGNED	HIGH	Cryptographically verifies software publisher and prevents binary tampering or trojanization. <i>Fix: Sign executable with trusted certificate via signtool.exe.</i>
Shannon Entropy Analysis	< 6.8	HIGH	Measures bit randomness. Values > 7.2 indicate encrypted, packed, or obfuscated payloads commonly found in droppers or malware. <i>Fix: Inspect packed resources or avoid commercial crypters.</i>

4. Android Package (.apk) Security Checks

Security Check	Default Safe	Risk if Present	What It Does & How to Fix
android:debuggable	false	CRITICAL	Allows JDWP debugger attachment, arbitrary memory dumping, and command execution via ADB on non-rooted devices. <i>Fix: Set <code>android:debuggable="false"</code> or use <code>release buildType</code> in Gradle.</i>
usesCleartextTraffic	false	HIGH	Permits transmission of credentials and API requests over unencrypted HTTP (vulnerable to MITM packet interception). <i>Fix: Set <code>android:usesCleartextTraffic="false"</code> & use HTTPS only.</i>
android:allowBackup	false	MEDIUM	Enables full ADB backup extraction of internal application storage, SQLite databases, and token caches. <i>Fix: Set <code>android:allowBackup="false"</code> in <code>AndroidManifest.xml</code>.</i>
Hardcoded Secrets & API Keys	None	CRITICAL	Scans <code>classes.dex</code> for embedded private keys, AWS tokens, JWT bearer secrets, Stripe keys, and database passwords. <i>Fix: Move sensitive secrets to secure backend or Google Secrets Manager.</i>
Exported Components	false	MEDIUM	Activities, receivers, or services with <code>android:exported="true"</code> can be invoked by any third-party malicious app. <i>Fix: Set <code>android:exported="false"</code> or require custom permission.</i>

AppGuard Hardening & Remediation

Remediation Workflow, AI Assistant Integration & Release Checklist

SECTION 5:
REMEDIATION

5. How to Remediate Issues Using the AI Prompt Box

After analyzing an app, AppGuard dynamically compiles a context-aware **AI Remediation Prompt** under the AI Prompt tab. This prompt contains:

- The exact list of discovered vulnerabilities with file offsets and configuration keys.
- Step-by-step code and build-configuration changes needed to remediate each finding.
- Security standards references (OWASP MASVS, Microsoft SDL, NIST SP 800-218).
- Zero-regression instructions ensuring application functionality remains intact.

Standard Remediation Loop:

1. Click **Copy Prompt** in the AppGuard UI.
2. Paste into your LLM assistant (Gemini, Claude, or ChatGPT) alongside your project's `build.gradle`, `CMakeLists.txt`, or `AndroidManifest.xml`.
3. Apply the generated patch files to your codebase.
4. Re-build the binary in **Release Mode**.
5. Re-upload the new binary to AppGuard to verify that the score drops to  **Green (≤ 3.9)**.

6. DevSecOps Production Readiness Checklist

Pre-Release Gate Item	Target Value	Responsible Tool / Flag
Composite Hackability Score	≤ 3.9 / 10	AppGuard Quantitative Gate
Critical Vulnerabilities	0 findings	Zero tolerance for production
Android Debug Mode	Disabled (false)	<code>buildTypes.release.debuggable</code> <code>false</code>
Android Network Security	TLS 1.2 / 1.3 only	<code>network_security_config.xml</code>
Windows Binary Exploit Mitigations	ASLR + DEP + SafeSEH	<code>MSVC</code> / <code>DYNAMICBASE</code> / <code>NXCOMPAT</code>
Binary Tamper Protection	Valid Authenticode / APK v3	Code Signing Certificate